

خواندن و نوشتن فایل در پایتون

باز کردن فایل‌ها در پایتون

پیش از آن که شروع به خواندن و نوشتن فایل‌ها بکنیم، نخستین گام این است که عملاً فایل را باز کنیم. به این منظور از تابع `open()` استفاده می‌کنیم که یک اشاره‌گر فایل ایجاد می‌کند. این اشاره‌گر در یک متغیر ذخیره می‌شود. ما باید روشی که قصد داریم با فایل تعامل پیدا کنیم را مشخص سازیم. چهار روش برای تعامل با یک فایل وجود دارند که شامل خواندن (`r`)، نوشتن (`w`)، الحاق (`a`) و ایجاد (`x`) می‌شوند.

```
1 filename = "app.log"
2 with open(filename, "r") as f:
3     print(f)
4     # <_io.TextIOWrapper name='app.log' mode='r' encoding='cp1252'>
```

ابتدا باید نام فایل را به یک متغیر به نام `filename` انتساب دهیم. سپس از کلیدواژه `with` همراه با تابع `open()` برای ایجاد اشاره‌گر خواندن فایل که به صورت `f` درون بلوک کد مورد اشاره قرار می‌گیرد، استفاده می‌کنیم. توجه کنید که پرینت کردن `f` موجب پرینت شدن فایل نمی‌شود، بلکه اطلاعات اشاره‌گر پرینت می‌شود. هدف از استفاده از `with` دور زدن مدیریت اشاره‌گر فایل است. فراخوانی مستقیم `open()` هیچ مشکلی ندارد، اما در ادامه باید زمانی که کارتان با فایل پایان یافت، اشاره‌گر را ببندید.

```
1 f = open("app.py", "r")
2 print(f)
3 f.close()
```

با استفاده از `with`، اشاره‌گر به صورت خودکار در انتهای بلوک بسته و کار ذخیره می‌شود.

خواندن فایل در پایتون

سه متد برای خواندن ژنریک یک فایل وجود دارد:

- متد `read()` کل فایل یا در صورت تعیین تعداد بایت، تا تعداد بایت مشخص شده را بازگشت می‌دهد.
- متد `readline()` یک خط منفرد یا در صورت تعیین تعداد بایت‌ها تا آن خط را بازگشت می‌دهد.
- متد `readlines()` کل فایل یا در صورت تعیین تعداد فایل، مقدار مشخص شده را بازگشت داده و به لیستی از خطوط افراز می‌کند.

اگر قصد دارید یک دستور منفرد مانند `print` را اجرا کنید، در این صورت متد `read()` به طور کلی کافی است. با این حال اگر قصد دارید با داده‌های خوانده شده کار کنید، در این صورت متد `readlines()` به تجزیه فایل کمک می‌کند:

```
1 with open("app.log", "r") as f:
2     print(f.read()) # will print the entire file
3
4     lines = f.readlines()
5     for line in lines:
6         print(line) # will print one line at a time
```

نوشتن فایل در پایتون

برخلاف خواندن فایل، دو حالت برای نوشتن یک فایل وجود دارد. اگر هدف شما از نوشتن فایل این است که با آن مانند یک فایل جدید رفتار کنید در این صورت از حالت نوشتن (`w`) استفاده کنید. این کار موجب پاک شدن همه محتوای موجود در فایل می‌شود. اگر می‌خواهید محتوای موجود را حفظ کنید در این صورت از حالت الحاق (`a`) استفاده کنید تا محتوای جدید در انتهای فایل نوشته شود.

دو متد برای نوشتن وجود دارد که شامل `write` و `writelines` است. از `write()` برای یک رشته منفرد و از `writelines()` برای یک لیست از رشته‌ها استفاده می‌کنیم. این متد مانند `readlines()` یک لیست از خطوط بازگشت می‌دهد:

```
1 one_line = "Hello World"
2 multiple_lines = ["Hello World", "Nice to Meet You"]
3 with open("app.log", "w") as f:
4     f.write(one_line)
5     f.writelines(multiple_lines)
```

فایل CSV در پایتون چیست؟

تمام [دانشمندان داده](#) (Data Scientists) باید روش خواندن اطلاعات از فایل‌های CSV در پایتون را بلد باشند. واژه CSV سرنامی از کلمات عبارت انگلیسی «Comma Separated Values» به معنای «مقادیر جدا شده با کاما» است. کاما به کاراکتر , گفته می‌شود CSV. فرمتی برای ذخیره‌سازی جدولی داده‌ها در قالب متن است. به عنوان دانشمند داده، هر روز و در امور کاری خود با فایل‌های CSV روبه‌رو می‌شویم. بنابراین مسلط شدن به توانایی خواندن و نوشتن فایل‌های CSV در پایتون برای مدیریت کارآمد و تحلیل صحیح مجموعه‌های داده، ضروری است.

دو روش کلی برای کار با فایل CSV در پایتون

برای کار با فایل‌های CSV در پایتون دو روش اصلی وجود دارند. این روش‌ها را در فهرست پایین معرفی کرده‌ایم.

- استفاده از ماژول «CSV»
- استفاده از کتابخانه «[Pandas](#)»

استفاده از ماژول CSV

ماژول CSV یکی از ماژول‌های درونی پایتون است. این ماژول با هدف کار بر روی فایل‌های CSV طراحی شده است. ماژول CSV تمام توابع ساده و اولیه مربوط به خواندن و نوشتن داده‌های CSV را فراهم کرده است.

در کدهای زیر، مثال ساده‌ای را درباره خواندن فایل CSV با استفاده از متد `csv.reader` بررسی کرده‌ایم.

```
1 import csv
2
3 # Open the CSV file in read mode
4 with open('data.csv', 'r') as csvfile:
5     # Create a reader object
6     csv_reader = csv.reader(csvfile)
7
8     # Iterate through the rows in the CSV file
9     for row in csv_reader:
10         # Access each element in the row
11         print(row)
```

خواندن فایل CSV در پایتون

روش‌های زیادی برای خواندن اطلاعات نوشته شده در فایل‌های CSV وجود دارد. مهم‌ترین این روش‌ها در فهرست زیر معرفی کرده‌ایم.

- خواندن فایل CSV با استفاده از متد `csv.reader`
- خواندن فایل CSV با استفاده از تابع `readlines()`
- خواندن فایل CSV با استفاده از ابزارهای کتابخانه `Pandas`
- خواندن فایل CSV با استفاده از متد `csv.DictReader`

استفاده از عبارت `with` برای پیاده‌سازی کد

قبل از استفاده از عبارت `with` لازم است که با سینتکس کامل این عبارت آشنا شویم. در کادر زیر این سینتکس را نوشته‌ایم.

```
1 with open(filename, mode) as alias_filename
```

در سینتکس بالا سه پارامتر وجود دارند. این پارامترها را در فهرست زیر معرفی کرده‌ایم.

- **filename:** نام همان فایلی است که می‌خواهیم بر روی آن کار کنیم.
 - **mode:** در این پارامتر، حالت باز شدن فایل را مشخص می‌کنیم. برای این پارامتر می‌توان چهار مقدار مختلف `r` و `w` و `a` و `+` را نوشت. این حالت‌ها را در ادامه توضیح داده‌ایم.
 - **alias_filename:** در این بخش نام مستعاری برای فایل باز شده انتخاب می‌کنیم.
- پارامتر `mode` می‌تواند شامل چهار حالت زیر شود.
- **r:** این مقدار یعنی اینکه فایل در حالت فقط خواندنی باز شود.
 - **w:** این مقدار یعنی اینکه اگر فایل داده شده وجود نداشت، فایلی به همان نام ایجاد شود. سپس می‌توانیم بر روی آن فایل اطلاعات بنویسیم.
 - **a:** از این مقدار برای اضافه کردن داده به محتوای موجود در فایل استفاده می‌کنیم.
 - **+:** این مقدار هم برای ساخت فایل جدید به کار برده می‌شود. بر روی فایل جدید هم می‌توان اطلاعات نوشت و هم اطلاعات را از روی آن خواند.

در کادر زیر، کدهای مربوط به خواندن فایل CSV با استفاده از عبارت `with()` نوشته شده است.

```
1 import csv
2 rows = []
3 with open("Salary_Data.csv", 'r') as file:
4     csvreader = csv.reader(file)
5     header = next(csvreader)
6     for row in csvreader:
7         rows.append(row)
8 print(header)
9 print(rows)
```

خواندن فایل CSV با استفاده از `readlines()`

در این بخش از مطلب، روش واکشی اطلاعات هدر و ردیف‌های فایل CSV را با استفاده از عبارت‌های `open()` و `with()` بررسی می‌کنیم. در این تکنیک از کتابخانه CSV استفاده نخواهیم کرد.

برای انجام این کار متد `readlines()` را به کار می‌بریم. این متد تمام خط‌های درون فایل را به صورت عناصر لیست برمی‌گرداند. هر خط در فایل به یکی از خانه‌های لیست تبدیل می‌شود. از آنجا که اولین ردیف در فایل CSV مربوط به نام ستون‌ها و اصطلاحاً هدر فایل است، بنابراین هدر در اندیس 0 لیست قرار می‌گیرد. بقیه عناصر لیست هر کدام از یک ردیف از داده‌های درون فایل CSV تشکیل شده‌اند.

در کدهای زیر روش استفاده از این تکنیک را برای خواندن داده‌های CSV بررسی کرده‌ایم.

```
1 with open('Salary_Data.csv') as file:
2     content = file.readlines()
3 header = content[:1]
4 rows = content[1:]
5 print(header)
6 print(rows)
```

نکته: شاید در انتهای هر خط از متن، کاراکترهای «`\n`» قرار گرفته باشند. با کمک متد `.strip()` می‌توانیم این کاراکترها را حذف کنیم.

نوشتن فایل CSV در پایتون

روش‌های مختلفی برای نوشتن داده بر روی فایل CSV در پایتون، وجود دارند. در این بخش از مطلب، چهار مورد از بهترین روش‌ها را معرفی کرده‌ایم.

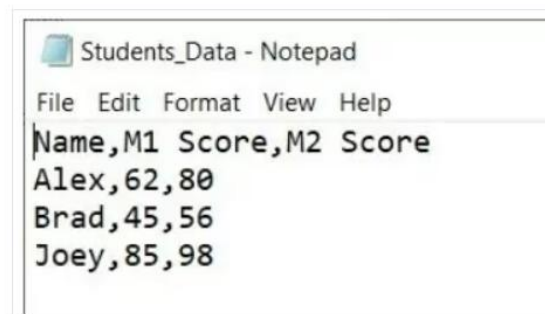
- نوشتن داده در فایل CSV با `csv.writer`
- نوشتن داده در فایل CSV با تابع `writelines()`
- نوشتن داده در فایل CSV با `Pandas`
- نوشتن داده در فایل CSV با `csv.DictWriter`

```
1 header = ['Name', 'M1 Score', 'M2 Score']
2 data = [['Alex', 62, 80], ['Brad', 45, 56], ['Joey', 85, 98]]
```

- مرحله اول: قبل از هر چیز باید کتابخانه CSV را به محیط کدنویسی خود وارد کنیم.
- مرحله دوم: برای فایل خود نامی را در نظر می‌گیریم. سپس با استفاده از تابع `open()` این فایل را باز می‌کنیم.
- مرحله سوم: با استفاده از کلاس `csv.writer()`، شیئی به نام `csvwriter` ایجاد می‌کنیم.
- مرحله چهارم: در شیء ساخته شده، هدر مورد نظر خود را می‌نویسیم.
- مرحله پنجم: بقیه داده‌های داده شده را با کمک کد `csvwriter.writerows(data)` می‌نویسیم.

```
1 filename = 'Students_Data.csv'
2 with open(filename, 'w', newline='') as file:
3     csvwriter = csv.writer(file) # 2. create a csvwriter object
4     csvwriter.writerow(header) # 4. write the header
5     csvwriter.writerows(data) # 5. write the rest of the data
```

بعد از اجرای کدهای بالا فایل CSV ساخته شده به شکل زیر، خواهد بود.



نوشتن داده در فایل CSV با تابع `writelines`

تابع `writelines()` بر روی لیستی پیمایش می‌کند. سپس تمام عناصر موجود در آن لیست را به رشته تبدیل می‌کند. همیشه قبل از ذخیره‌سازی داده‌ها در فایل CSV باید آن‌ها را به رشته تبدیل کنیم. تابع `writelines()` بعد از تبدیل کردن همه عناصر لیست به رشته، آن‌ها را در فایل CSV ذخیره می‌کند.

در کدهای زیر، روش کار این تابع نوشته شده است.

```

1 header = ['Name', 'M1 Score', 'M2 Score']
2 data = [['Alex', 62, 80], ['Brad', 45, 56], ['Joey', 85, 98]]
3 filename = 'Student_scores.csv'
4
5 with open(filename, 'w') as file:
6     # Create a list of strings to write
7     lines = []
8     # Add the header line
9     lines.append(','.join(str(item) for item in header) + '\n')
10    # Add each row of data
11    for row in data:
12        lines.append(','.join(str(item) for item in row) + '\n')
13
14    # Write all lines at once
15    file.writelines(lines)

```

تمام مراحل انجام شده در کدهای بالا را در فهرست زیر نوشته‌ایم.

1. در خط اول هدر فایل CSV را تعریف کرده‌ایم.
2. در خط دوم داده‌هایی را تعریف کرده‌ایم که باید در فایل ذخیره شوند.
3. در خط ۳ نام فایل را مشخص کردیم.
4. در خط ۵ فایل مورد نظر را در حالت 'w' برای نوشتن باز کردیم.
5. در خط ۷ لیست خالی lines را تعریف کردیم. ابتدا تمام داده‌ها را به رشته تبدیل کرده و در این لیست ذخیره می‌کنیم.
6. سپس در خط ۹ داده‌های مربوط به هدر فایل CSV را به رشته تبدیل کرده و به لیست lines اضافه کردیم.
7. از خط ۱۰ تا خط ۱۱ هم داده‌های مورد نظر را به لیست lines اضافه کردیم.
8. در خط ۱۵ تمام عناصر ذخیره شده در لیست lines را با کمک دستور file.writelines(lines) در فایل CSV نوشتیم.

JSON برای پایتون

جیسون، عبارت اختصاری برای «نمادگذاری اشیای جاوا اسکریپت (JavaScript Object Notation)» و یک قالب مبتنی بر متن است که امکان تبادل داده‌ها بین برنامه‌های مختلف را تسهیل می‌کند. برای نمونه یک برنامه‌ای که در C++ نوشته شده است و روی ویندوز اجرا می‌شود، می‌تواند به راحتی با استفاده از جیسون داده‌ها را با برنامه‌ای در پایتون نوشته شده و بر روی لینوکس اجرا می‌شود، مبادله کند.

شیء در واقع یک مجموعه از نگاشت «کلید: مقدار» است که به شکل زیر نمایش می‌یابد:

```
person.json:
{
  "firstName": "Alice",
  "lastName": "Hall",
  "age": 35
}
```

در ادامه شیوه نمایش یک آرایه از اشیاء را مشاهده می‌کنید. در این نمایش هر آیتم در آرایه در واقع یک شیء است. این نمونه‌ای از حقوق بازیکنان بیس‌بال است.

```
salaries.json:
[
  {
    "year": 1985,
    "teamId": "ATL",
    "leagueId": "NL",
    "playerId": "barkele01",
    "salary": 870000
  }, {
    "year": 1985,
    "teamId": "ATL",
    "leagueId": "NL",
    "playerId": "bedrost01",
    "salary": 550000
  } ]
```

تجزیه جیسون در پایتون

پایتون یک ماژول جیسون دارد که از آن هم برای تجزیه و هم تولید جیسون از اشیای پایتون و لیست‌ها می‌توان استفاده کرد.

در ادامه قطعه کدی مشاهده می‌کنید که روش باز کردن یک فایل جیسون و بارگذاری داده‌ها درون یک متغیر را نشان می‌دهد.

```
import json
with open('sample.json', 'r') as fp:
    obj = json.load(fp)
```

وقتی رشته‌ای حاوی داده‌های جیسون دارید می‌توانید آن را با استفاده از کد زیر به یک شیء (یا لیست) پایتون تبدیل کنید:

```
obj = json.loads("""{
"firstName": "Alice",
"lastName": "Hall",
"age": 35
}""")
```

مدیریت خطاها

وقتی جیسون با خطا مواجه شود یک مقدار ValueError دریافت می‌کنید. می‌توان با انجام یک اقدام اصلاحی این خطا را مدیریت کرد.

```
try:
    obj = json.loads("""{
"firstName": "Alice",
"lastName": "Hall",
"age": 35
}""")
except ValueError:
    print "error loading JSON"
```

دسترسی به داده‌ها

زمانی که داده‌های جیسون درون متغیر پایتون لود شدند، می‌توانید داده‌ها را به هر صورتی که در پایتون دوست دارید (و یا حتی به صورت لیست) در آورید. برای مثال داده‌های جیسون فوق به صورت زیر در پایتون قابل دسترسی هستند:

```
firstName = obj["firstName"]  
lastName = obj["Hall"]  
age = obj["age"]
```